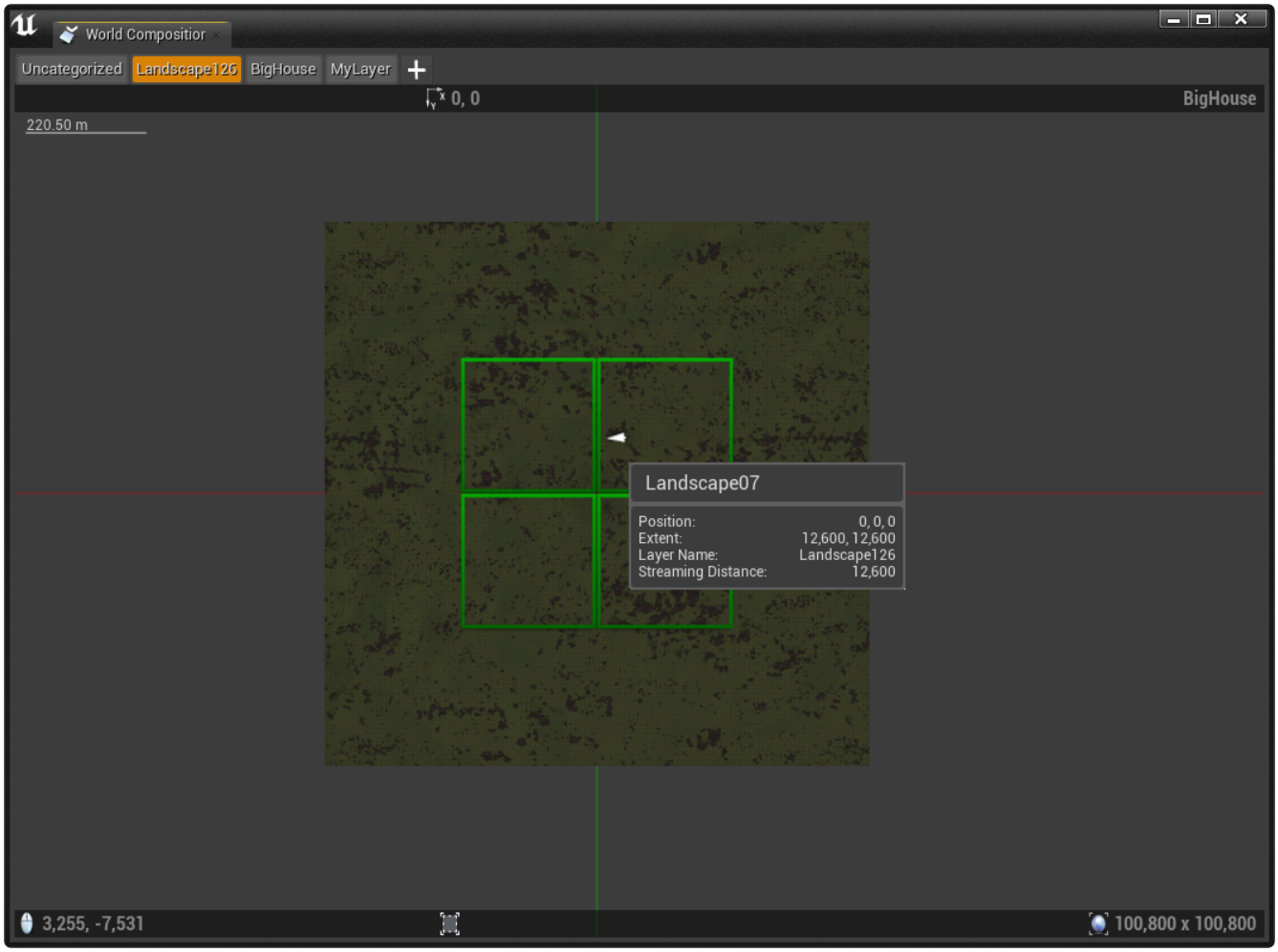
大地图优化方案

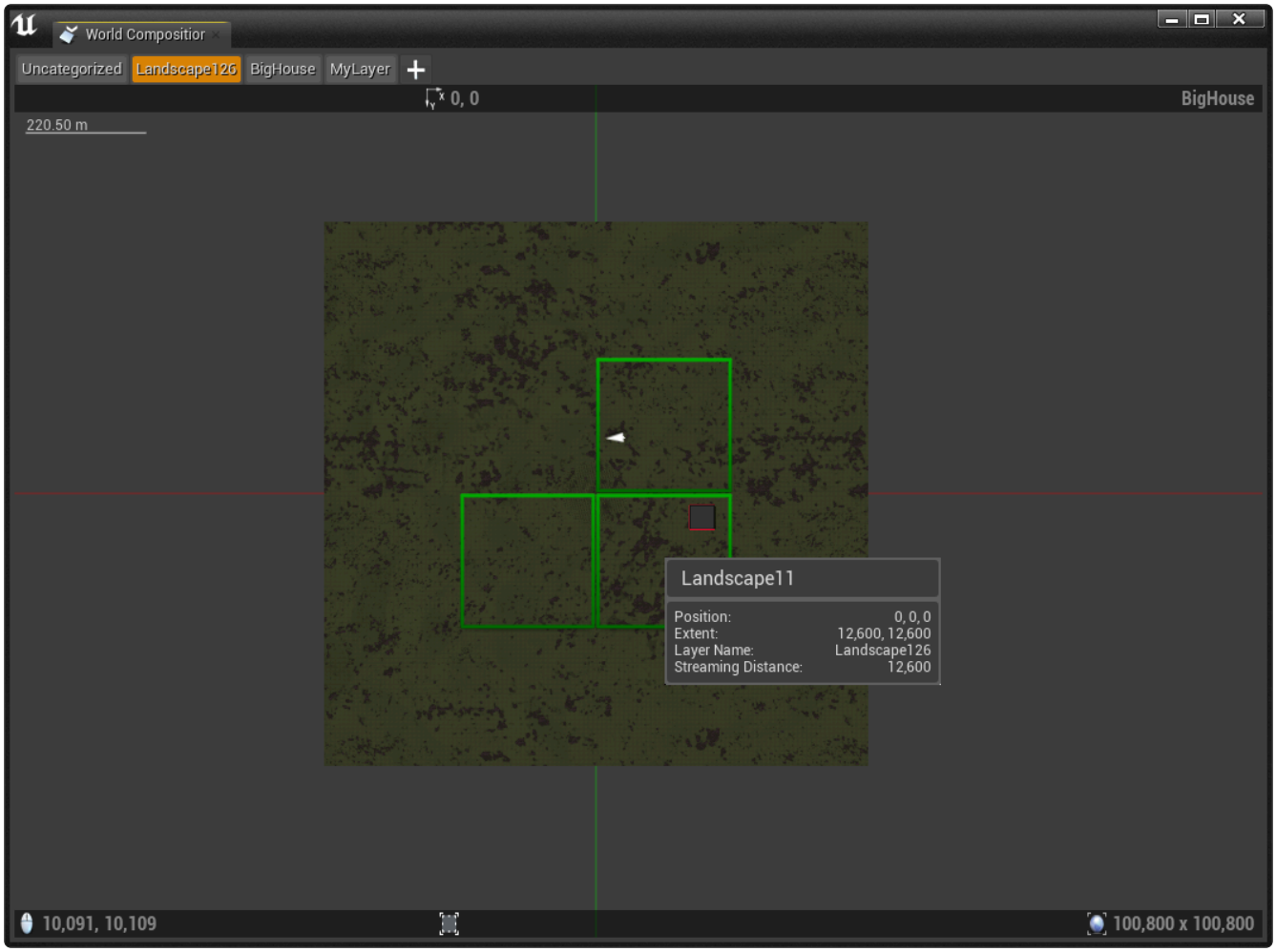
一、World Composition

World Composition（后面简称 WC）是 UE 提供的进阶的大地图实现的技术方案，基本上可以解决 LevelStreamingVolume 所面临的所有问题

1.1 特点

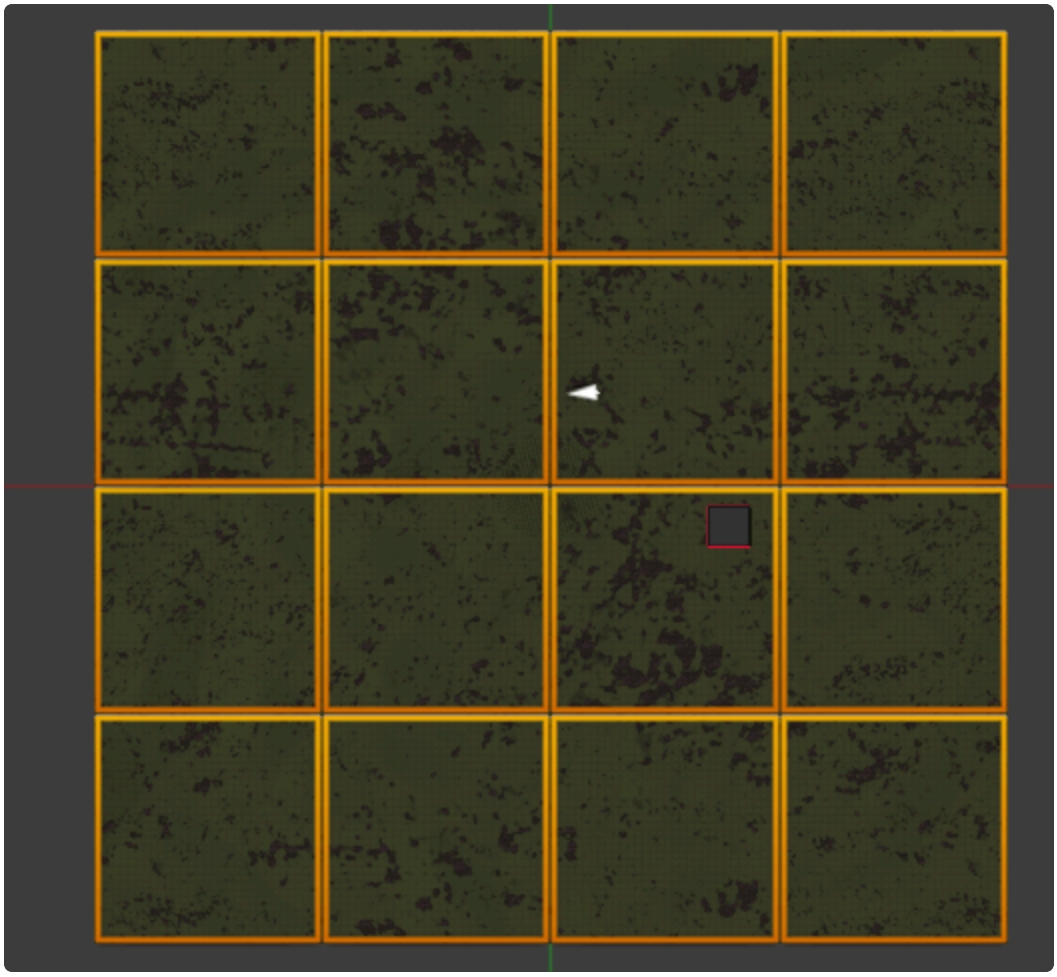
- **制作简单且可预览效果**





> TODO：目前WC编辑器存在一些操作繁琐的地方，后续技术方案确定后我会在引擎底层尝试扩展一下工具功能，这个应该不会太麻烦

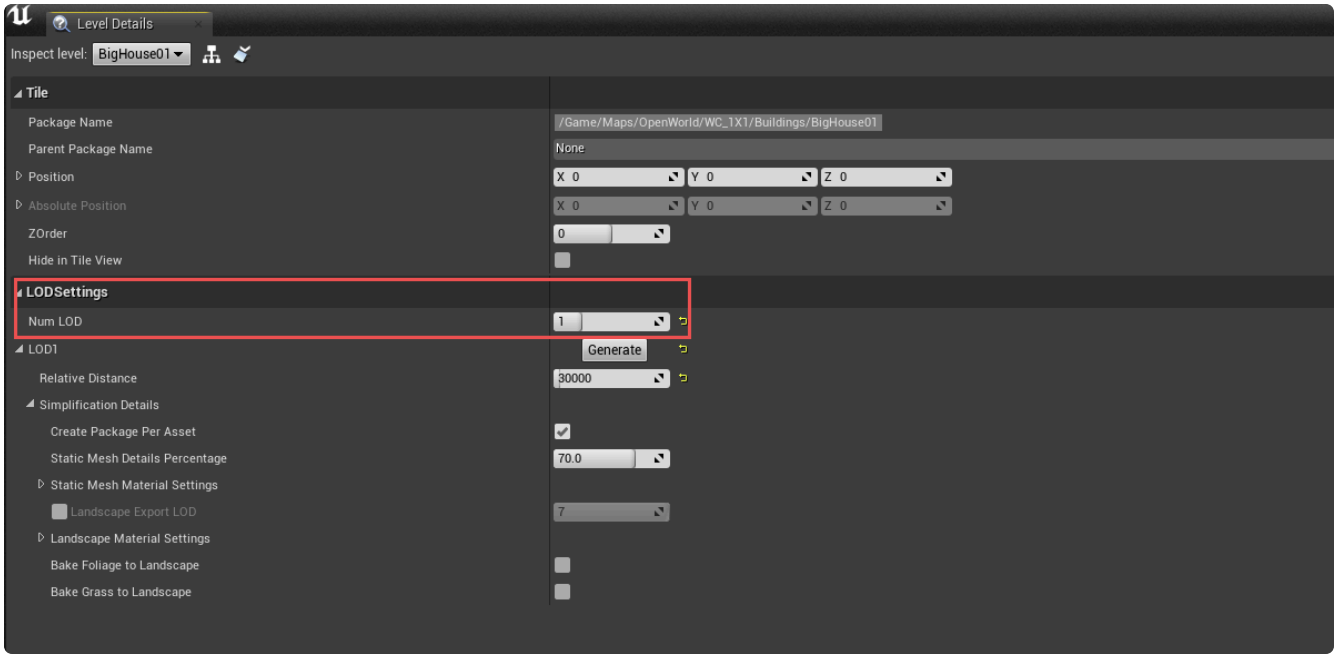
- **空间分区**



- **分层**

- **LOD**

WC 模式下，支持 LevelStreaming 的 LOD，保证游戏显示效果的同时还能兼顾性能



> 使用Level LOD会使包体增大，内存也会有一定增幅
> 因此具体是否使用可以再权衡一下，实际上如果不使用LOD，美术制作关卡时可以达到对应的效果，但是会对场景设计有一定约束，以保证子关卡切换不会穿帮

- **边界切换有 CD 优化**

C/C++

```
bool UWorldComposition::CommitTileStreamingState(UWorld* PersistentWorld, int32 TileIdx, bool bShouldBeLoaded, bool bShouldBeVisible, bool bShouldBlock, int32 LODIdx)
{
    ......

    // Quit early in case we have cooldown on streaming state changes
    const bool bUseStreamingStateCooldown = (PersistentWorld->IsGameWorld() && PersistentWorld->FlushLevelStreamingType == EFlushLevelStreamingType::None);
    if (bUseStreamingStateCooldown && TilesStreamingTimeThreshold > 0.0)
    {
        const double CurrentTime = FPlatformTime::Seconds();
        const double TimePassed = CurrentTime - Tile.StreamingLevelStateChangeTime;
        if (TimePassed < TilesStreamingTimeThreshold)
        {
            return false;
        }

        // Save current time as state change time for this tile
        Tile.StreamingLevelStateChangeTime = CurrentTime;
    }

    ......
}
```

> 目前主流的技术方案是CD+加距离缓冲，可以更好避免边界频繁切换导致的性能问题

- **联机同步**

支持 ServerTravel 和 ClientTravel 同步

> TODO：具体待深入研究

- **灵活性**

WC 是完全受引擎底层托管的，因此灵活性比 LSV 差一些

但引擎底层也是提供了 API 方便自定义关卡管理

C/C++

```cpp
UCLASS(BlueprintType)
class ENGINE_API ULevelStreamingDynamic : public ULevelStreaming
{
    ......

    /**
     * Stream in a level with a specific location and rotation. You can create multiple instances of the same level!
     *
     * The level to be loaded does not have to be in the persistent map's Levels list, however to ensure that the .umap does get
     * packaged, please be sure to include the .umap in your Packaging Settings:
     *
     *   Project Settings -> Packaging -> List of Maps to Include in a Packaged Build (you may have to show advanced or type in filter)
     *
     * @param LevelName - Level package name to load, ex: /Game/Maps/MyMapName, specifying short name like MyMapName will force very slow search on disk
     * @param Location - World space location where the level should be spawned
     * @param Rotation - World space rotation for rotating the entire level
    * @param bOutSuccess - Whether operation was successful (map was found and added to the sub-levels list)
    * @param OptionalLevelNameOverride - If set, the loaded level package have this name, which is used by other functions like UnloadStreamLevel. Note this is necessary for server and client networking because the level must have the same name on both.
     * @return Streaming level object for a level instance
     */
     UFUNCTION(BlueprintCallable, Category = LevelStreaming, meta=(DisplayName = "Load Level Instance (by Name)", WorldContext="WorldContextObject"))
     static ULevelStreamingDynamic* LoadLevelInstance(UObject* WorldContextObject, FString LevelName, FVector Location, FRotator Rotation, bool& bOutSuccess, const FString& OptionalLevelNameOverride = TEXT(""));
```

```

    UFUNCTION(BlueprintCallable, Category = LevelStreaming, meta=(DisplayName = "Load Level Instance (by Object Reference)", WorldContext="WorldContextObject"))
    static ULevelStreamingDynamic* LoadLevelInstanceBySoftObjectPtr(UObject* WorldContextObject, TSoftObjectPtr<UWorld> Level, FVector Location, FRotator Rotation, bool& bOutSuccess, const FString& OptionalLevelNameOverride = TEXT(""));

    ......
};
```

1.2 性能

尽管引擎底层没有采用传统更优的空间管理算法，但由于做好了空间分区和分层，可以保证只加载与玩家具有空间相关性的区域，因此能大大提高性能

1.3 可能造成的影响

- 美术

美术的工作流会有所变化，需要按照 WC 规则（分区 / 分层）来制作关卡和美术资源

- 策划

策划需要学习 WC 编辑器（这边可以提供简单的培训）

二、备选方案

LSV 和 WC 都是 UE 提供的技术方案，尽管 WC 相比 LSV 更具优，但还是存在一些不足之处：

- 代码层未做分区和分层管理

引擎底层，区块是统一管理的，虽然提供分区和分层功能，但区块未做分区和分层管理

C/C++

```
    // List of all tiles participating in the world composition
    FTilesList                    Tiles;
```

- 性能瓶颈

地图很大（开放世界），场景特别复杂时，区块和分层就会较多，此时暴力遍历所有区块可能存在一定性能瓶颈

C/C++

```cpp
void UWorldComposition::GetDistanceVisibleLevels(
    const FVector* InLocations,
    int32 NumLocations,
    TArray<FDistanceVisibleLevel>& OutVisibleLevels,
    TArray<FDistanceVisibleLevel>& OutHiddenLevels) const
{
    const UWorld* OwningWorld = GetWorld();

    FIntPoint WorldOriginLocationXY = FIntPoint(OwningWorld->OriginLocation.X, OwningWorld->OriginLocation.Y);

    for (int32 TileIdx = 0; TileIdx < Tiles.Num(); TileIdx++)
    {
        const FWorldCompositionTile& Tile = Tiles[TileIdx];

        // Skip non distance based levels
        if (!IsDistanceDependentLevel(TileIdx))
        {
            continue;
        }

        ......

        if (bIsVisible)
        {
            OutVisibleLevels.Add(VisibleLevel);
        }
        else
        {
            OutHiddenLevels.Add(VisibleLevel);
        }
    }
}
```

> 假设场景有N块，M层，引擎默认遍历的时间复杂度就为O（N+M）

> 当然这种情况理论上也是可以优化的：
> 利用三维数组，先定位Layer，再定位Tile，可以做到O（1）的时间复杂度

- StreamingLevel 边界处理

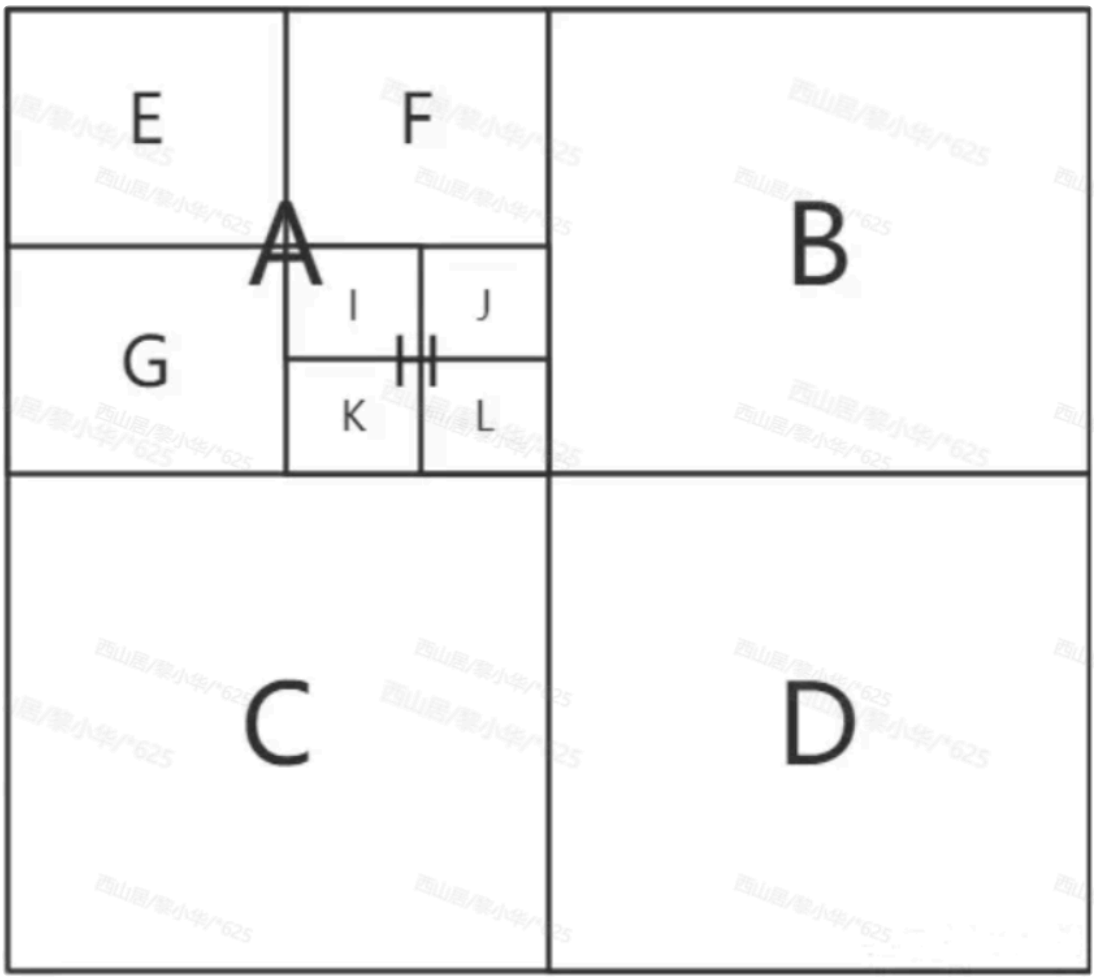
如果某个建筑物在两个或多个 StreamingLevel 边界处

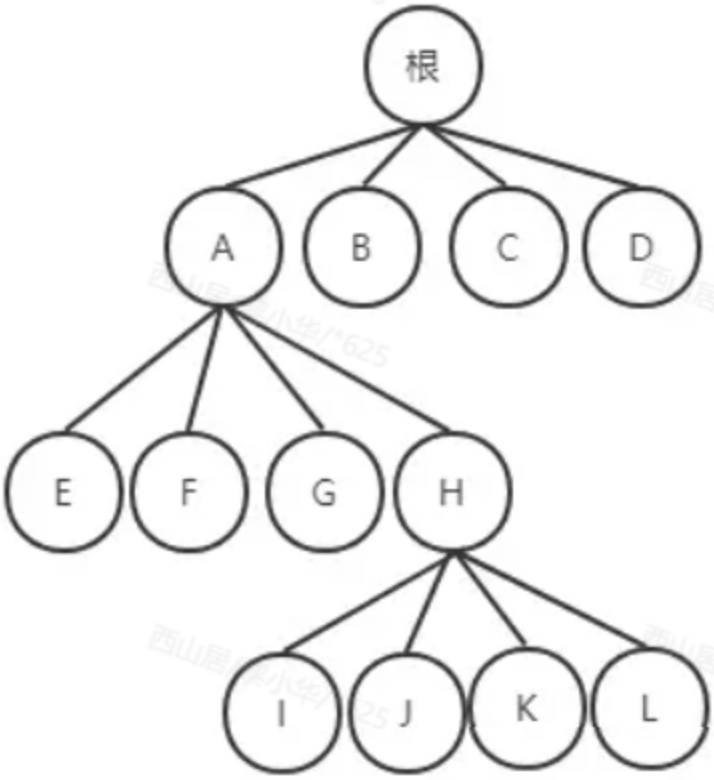
- 需由引擎托管，不够自由化

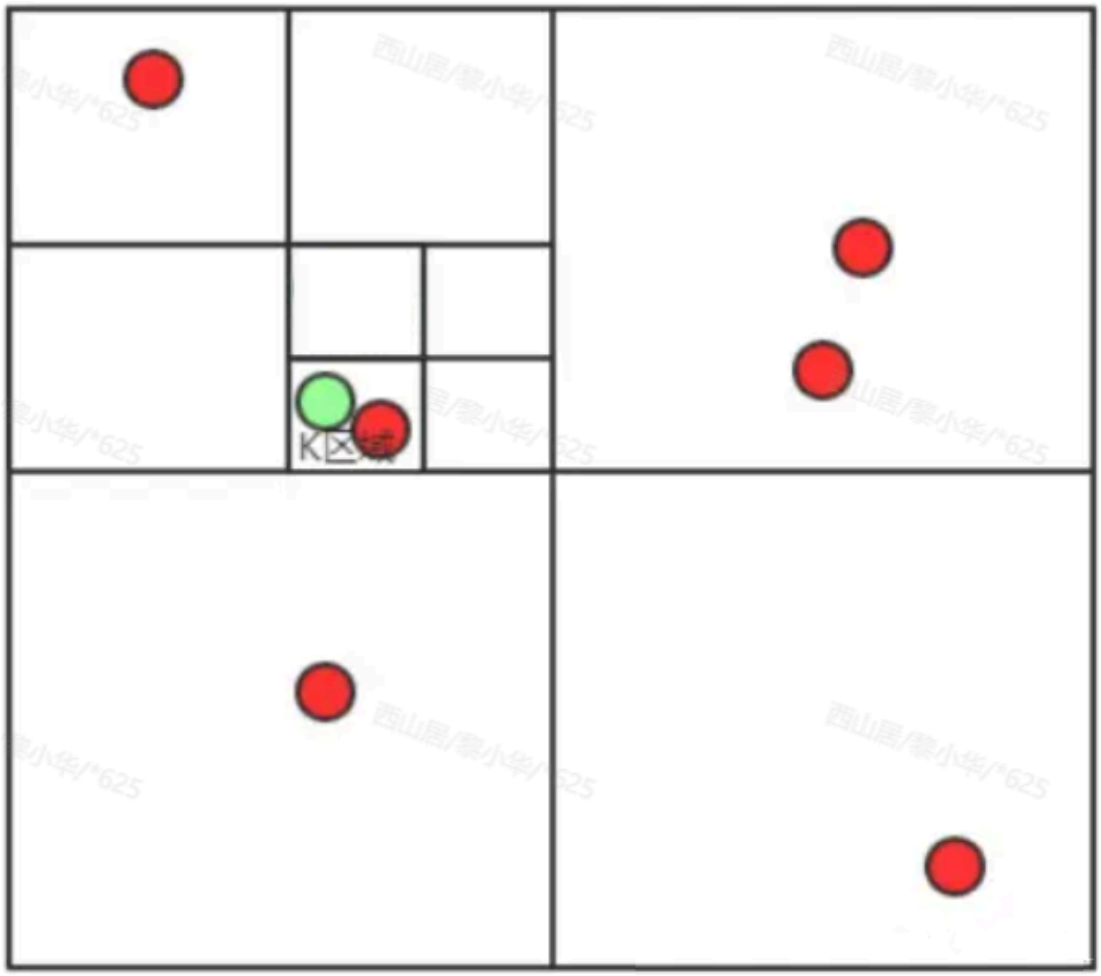
如果需要高度自定义管理所有的 StreamingLevel，WC 可能就不太适合了

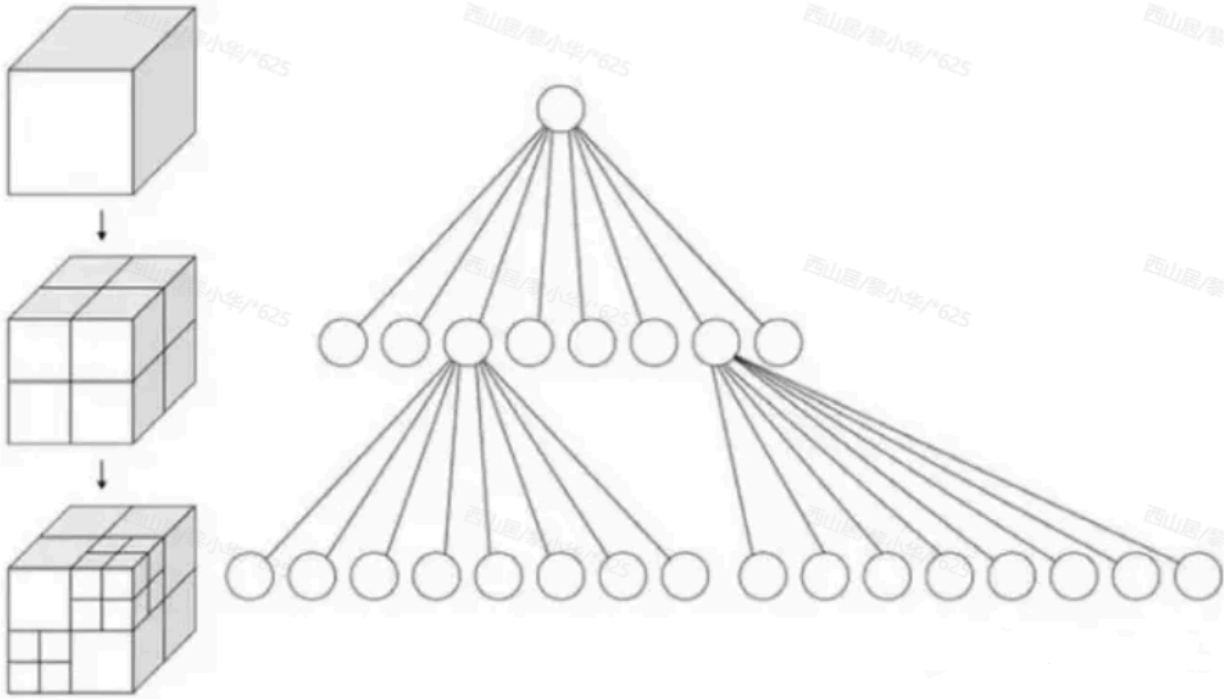
2.1 QuadTree

即四叉树，2D 平面管理算法，适合没有 HeightMap 或没有 Z 轴变换的大地图




根
A
B
C
D
E
F
G
H
I
J
K
L



2.2 Octree

即八叉树，3D 空间管理算法，相比四叉树多了一个维度，适合有 HeightMap 或有 Z 轴变换的大地图